
Observing Knowledge at Structural Interfaces

*An Iterative Design Case for
Automated Knowledge-Capture Support*

Author: Elizabeth Osborn

Date: May 10, 2026

Abstract

Knowledge generation in modern technical organizations increasingly outpaces inherited methods for retaining and transferring it. Documentation, mentorship, and succession planning assume that knowledge can be stabilized and recorded after work is complete, but this assumption weakens in rapid, distributed, technically dense environments where knowledge emerges during execution and disperses with the people who produce it. This paper presents a design case for an iterative, vendor-agnostic knowledge-capture support method developed inside a multi-university CubeSat software program. The artifact uses automated extraction, staged review, observability, and a live knowledge graph to capture candidate knowledge from working artifacts while the work is still in motion. The contribution is the layered architecture and task structure that allow automation to support knowledge retention without binding the system to a specific vendor, model, or implementation stack. The current operating-state design uses human verification to stabilize entries before they enter the verified knowledge graph, while the broader architecture points toward more deeply embedded agent infrastructure, richer observability, and automated correction loops. By separating replaceable infrastructure from the research-bearing layers of mapping and metadata analysis, the method can evolve with artificial intelligence infrastructure while continuing to address a persistent organizational problem: retaining, verifying, and reusing technical knowledge generated faster than conventional transfer practices can capture it.

Contents

1	Introduction	4
2	Constraints	5
2.1	Users	5
2.1.1	Primary user: Operator	5
2.1.2	Secondary user: Beneficiary	5
2.1.3	Tertiary user: Supporting organization	5
2.2	System Operating Conditions and Environment	5
3	Methodology	8
3.1	Task Breakdown and Information Flow	8
3.1.1	Task 1: Connect a Source	9
3.1.2	Task 2: Review and Verify Extractions	10
3.1.3	Task 3: Compare Verified Knowledge to Agent Health and Recalibrate . .	11
3.2	Decision Gates Across Three Levels	13
3.3	User-System Interaction Model	13
3.4	Project Design	16
3.4.1	Iterative Design Process	16
3.4.2	Final Operating-State Design	18
3.4.3	Tools Used to Develop the Design	22
3.5	Design walkthrough and evaluation	22
3.5.1	Constraint-Based Walkthrough	22
3.5.2	Pilot Status	23
3.5.3	Real-World Performance Risks	23
3.5.4	Future Empirical Evaluation Methods	24
4	Conclusion	25
4.1	What the project process showed	25
4.2	What more time, money, and resources would have allowed	25
4.3	How the project design process would be improved	25
4.4	Closing	27
A	FRAMES Prototype Screenshots	27

List of Figures

1	Top-level HTA diagram.	8
2	HITL trajectory across the three tasks.	9
3	Disposition gate state diagram.	11
4	Task 3 compare-and-recalibrate flow.	12
5	Information architecture of the application's surfaces.	14
6	Operator-system sequence diagram.	15
7	The iteration gate firing across seven construction attempts.	16
8	Operating-state layered architecture.	18
9	Admin operations dashboard.	19
10	Mission Control live knowledge graph.	20
11	Review Queue surface.	20
12	Two-pass verification gate.	21
13	Knowledge Library surface.	21
14	Architectural improvement vision.	26
15	FRAMES dashboard entry surfaces.	28
16	FRAMES dashboard analysis surfaces.	28
17	FRAMES Research Tools and Analysis surface.	29

List of Tables

I	Operator requirements.	6
II	Beneficiary requirements.	6
III	Supporting organization requirements.	7
IV	System operating constraints.	7
V	Acknowledged limitations.	8
VI	Task 1: information flow, system feedback, completion cue.	10
VII	Task 2: information flow, system feedback, completion cue.	10
VIII	Task 3: information flow, system feedback, completion cue.	12
IX	Decision gates across three levels.	13
X	Iterative design process.	17
XI	Operating-state system architecture.	18
XII	Tools used to develop the design.	22
XIII	Design factors and pilot status.	23
XIV	Real-world performance risks.	24
XV	Research methods for evaluation.	24

1 Introduction

In late 2025, a team of engineers from multiple university space labs shared a mission to create flight software for CubeSat hardware each space lab would attempt to build and launch. By early 2026, the team was successful and had begun moving from their academic organizations into the commercial aerospace industry. The instrument used to observe and retain the knowledge they created was still in its infancy. The repositories they left behind are still easily accessible online but the knowledge of how to interpret them is difficult to recover.

What entered the digital record was the repositories, pull requests, design documents, and configuration files. What did not enter the record was the understanding of how the system was built, why specific design decisions were made, which interfaces were fragile, and which couplings depended on a specific person’s continued attention. New students or contributors coming into the lab today can find the technical information; they will not find the people who created it. Technical documentation can hold only certain types of knowledge, and the rest is lost.

The team struggled to capture the knowledge they generated because they were: (1) generating new knowledge through execution, rather than reconstructing it after the fact; (2) producing across distributed teams operating semi-independently; (3) creating highly technical content; and (4) operating at rapid speed, all of which make knowledge retention and transfer nearly impossible. These conditions are not unique to university CubeSat teams or aerospace research. Modern industrialization, sometimes referred to as the Fifth Industrial Revolution [1] (5IR), has created an environment where knowledge is generated faster than current transfer methods, including documentation, mentorship, and succession planning, can handle. There is now a mismatch between the technologies used to create knowledge and the technologies used to preserve it. The instrument developed alongside the team is what addresses the misalignment.

Knowledge loss is currently studied through organizational theory [2] and knowledge management [3], yet these two fields alone are not equipped to create the lens needed to observe the boundary between humans and technology. A technology domain defines its own boundaries against the domains it understands less well, and a human domain does the same. Domains that study human-environment interaction have the structural tools to understand this interface. The instrument developed to collect the knowledge being lost during the rapid development of the university space program is therefore examined as a sociotechnical structure: how the design supports automation as an interface between humans and a system, and how the automation and system shape how humans view accountability for knowledge loss.

This paper contributes a design case for an iterative, vendor-agnostic knowledge-capture support method developed inside a distributed university CubeSat program. The contribution is the task model, decision-gate structure, and layered architecture that separate replaceable AI infrastructure from the research-bearing layers of mapping and metadata analysis. The paper does not present a finished evaluation of long-term system performance. It presents the architecture, operating-state design, and verification structure that emerged from iterative construction under rapid, distributed, technically dense work conditions.

2 Constraints

2.1 Users

The system has three users in distinct positions relative to the work. The operator is the engineer using the system as a tool in their toolkit and interacts with the system most directly. The beneficiary is the next cohort entering the lab or technical team, who inherits the verified library after the operator's work is done. The supporting organization is the academic institution, knowledge retention group, mission organization, or onboarding team that holds the structure together across personnel transitions and carries the cost when transitions fail.

2.1.1 Primary user: Operator

The operator is the engineer or subject matter expert within a distributed technical organization, responsible for the physical components from which knowledge is being captured. The operator is the same engineer who is doing the technical work, with the system as one tool in their working toolkit.

2.1.2 Secondary user: Beneficiary

The beneficiary is the next cohort entering the lab or technical team, into a structure where knowledge transfer between cohorts is the primary determinant of program survival [3]. The beneficiary typically has no living institutional memory of how the work was originally built. The beneficiary does not interact with the agents and does not see the Observatory at operator time; the verified library is what they receive.

2.1.3 Tertiary user: Supporting organization

The supporting organization is the entity that holds the structure inside which the work happens: academic institutions whose professors carry teaching knowledge their successors will need to build curriculum, mission organizations whose engineers carry knowledge whose loss can cause catastrophic failure on the next mission, knowledge retention groups, onboarding teams, or any organization where knowledge loss is an organizational-scale risk. The supporting organization does not perform the technical work and does not consume the library to do it. The supporting organization holds the structure inside which the operator and the beneficiary meet.

2.2 System Operating Conditions and Environment

The conditions on the operator and the conditions on the system are the same conditions, seen from opposite sides of the interface between them, each creating data directly for the other at a pace the other is unable to process procedurally. The tables below organize those conditions into the requirements they place on the system (grouped by which user places each), the constraints they impose on how the work happens, and the limitations they impose on what any system in this class can do.

Table I: Operator requirements: what the engineer using the system needs from it in order to keep using it during live technical work.

Requirement	Conditions and environment	Strengths and allowances	Assumptions
Quick and easy to use	Engineers will not stop work to operate a system that adds friction.	Agentic AI handles the slow parts of reading sources and drafting extractions, keeping the operator's contact with the system light.	The operator's time is the scarcest resource in the workflow.
Captures knowledge faster than the operator can write it down	Knowledge emerges through execution, faster than any individual operator can record it.	Agents read source artifacts in parallel with the engineering work, at the pace the work is produced.	Agent extractions are accurate enough to review rather than rewrite.
Captures knowledge the operator does not know how to record	Embodied knowledge cannot be written down in the form documentation assumes; the operator may know it without being able to encode it.	Agents map where embodied knowledge resides at structural addresses, even when the content cannot be captured.	The structural addresses are readable by people other than the operator.
Easy to navigate between deploying, observing, and reviewing	The operator moves between three operational surfaces during a working session.	A unified dashboard holds all three surfaces in one place, with consistent shape and navigation.	The operator works in sessions, not in continuous attention.
Lets the operator see how the system is performing behind the scenes	The operator is accountable for what enters the verified library; visibility is the precondition for that accountability.	Observability infrastructure carries traces of model calls, agent decisions, and aggregate dashboards across the running system.	The operator can interpret what the observability surfaces show.

Table II: Beneficiary requirements: what the next cohort needs from the verified library when they inherit it.

Requirement	Conditions and environment	Strengths and allowances	Assumptions
Holds what previous generations knew	Original contributors are gone; the library is the surviving record.	Verified knowledge stored as a graph can be reached through relationships, not only through names the beneficiary may not know.	The library contains enough of what previous generations knew to be worth pulling from.
Can be interrogated easily	A beneficiary in a new cohort cannot pause onboarding to learn a complicated tool.	The same dashboard the operator works against carries the library, with role-differentiated access for the beneficiary.	The beneficiary knows enough about the work to know what to ask.
More useful than the repositories of the past	Repositories preserve what was written; embodied knowledge that could not be written is what previous generations of repositories lost.	Agentic mapping can locate embodied knowledge at structural addresses that a repository cannot represent.	The mapping is precise enough for the beneficiary to act on what it surfaces.
Returns information quickly	The beneficiary is doing live work; long retrieval slows the cohort's onboarding.	Relational graph technology is mature enough to power library traversal at interactive speeds.	The library is reachable through a query path that does not require the beneficiary to know the structure of the data.

Table III: Supporting organization requirements: what the institution that holds the structure across personnel transitions needs from the library.

Requirement	Conditions and environment	Strengths and allowances	Assumptions
Captures contributor knowledge that becomes an organizational asset	Contributors leave (professors retire, engineers move on, students graduate); the knowledge they held is what the organization needs to retain.	Agents extract from working artifacts at the pace the work is being done, before the contributor departs. A professor’s teaching knowledge feeds the next cohort’s curriculum; a mission engineer’s knowledge feeds the next mission’s safety.	The contributor’s working artifacts contain enough of what they know to be worth extracting.
Reduces the cost of knowledge loss at organizational scale	Knowledge loss accumulates as an organizational liability: curricula go stale, missions repeat failures, programs lose continuity across cohorts.	The library aggregates verified knowledge across contributors and across time, so the cost of a single departure is bounded. Catastrophic failures attributable to knowledge loss carry consequences that fall on the supporting organization, not on the departed contributor.	The library captures enough of what was at risk to materially reduce the cost when individual contributors depart.
Visible across the organization, not only to the operator	The organization must see what is being captured to verify its own continuity, not depend on the operator to report it.	A dashboard with workspace-level views shows the verified library across all contributing parts of the organization. Different parts of the organization need to see their own knowledge alongside the shared library.	The library’s structure can hold workspace-level distinctions without losing relational completeness across the whole.
Durable across the timescale the organization operates on	The organization’s timescale (decades for an academic institution, mission lifetimes for a space program) is longer than any contributor’s tenure.	The library is the artifact the organization carries forward; it must survive any specific cohort, mission, or curriculum revision.	The library outlasts the people who populated it.

Table IV: System operating constraints: the conditions the design must hold up under during live, distributed, technical work.

Constraint	Conditions and environment	Strengths and allowances	Assumptions
Knowledge emerges during execution	Knowledge is produced through doing the work, while the system is still changing; the pace of work will not slow to make capture easier.	Agentic AI reads source artifacts at the pace they are produced.	The work’s pace will not slow to make capture easier.
Distributed teams operating semi-independently	Geographic distance and different institutional tools; each institution has its own access patterns, so role and membership differentiation must be built in.	A shared dashboard carries visibility across institutions without forcing a shared toolchain.	The institutions continue to operate semi-independently.
Highly technical content	Incorrect extraction propagates faulty knowledge into future cohorts.	Domain-specific classification at extraction preserves technical type while source context is in hand.	The operator can verify technical accuracy at review.
Rapid speed of work	Any review process that slows engineers is abandoned in practice; there is no single window when engineers can be asked to wait.	Agentic extraction runs in the background; observability allows asynchronous review.	The operator can review asynchronously.

Table V: Acknowledged limitations: what no system in this class can do, and how the design works inside those boundaries.

Limitation	Conditions and environment	Strengths and allowances	Assumptions
Embodied knowledge cannot be captured directly in content form	The four constraints above concentrate the knowledge in embodied form; departing contributors carry the embodied knowledge most at risk.	Agentic AI cannot capture embodied knowledge as content but can map where it resides at structural addresses.	Embodied knowledge can be located at a structural address even when it cannot be written down.
Criticality cannot be assigned at moment-of-capture	What looks unimportant when captured often becomes critical later.	Capture is exhaustive at low cost; the criticality call moves to the operator at review time.	The operator at review time has enough context to assign criticality the agent could not.
Classification cannot be recovered after-the-fact	Technical types depend on source context that is not always available later; classification errors propagate forward, and downstream agents inherit them.	Agents classify at extraction while source context is in hand.	Classification at extraction is accurate enough that review is correction, not reconstruction.

A system that captures only codified knowledge is a data extractor. The requirements and the limitations together describe the gap between data extraction and knowledge mapping that the rest of the paper addresses.

3 Methodology

This methodology uses hierarchical task analysis, user-system interaction modeling, iterative design documentation, and constraint-based evaluation to describe how the proposed knowledge-capture system was developed and assessed. The focus is the operator’s interaction with the system because the operator is the user who connects sources, observes agent activity, and verifies candidate extractions before they enter the Knowledge Library.

3.1 Task Breakdown and Information Flow

The operator’s superordinate goal is to capture knowledge from sources into the verified Knowledge Library before that knowledge disperses with the people who hold it. Reaching the goal requires three primary tasks: (1) connect a source, (2) review and verify extractions, and (3) compare verified knowledge to agent health and recalibrate.

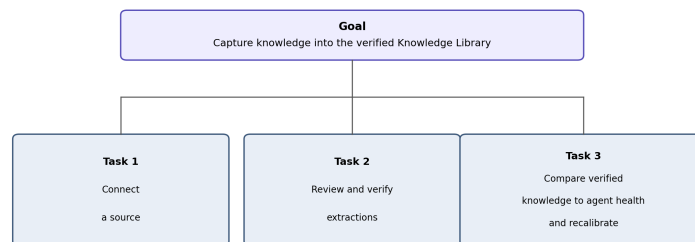


Figure 1: Top-level diagram. Goal: capture knowledge from sources into the verified Knowledge Library. Three tasks below the goal: (1) connect a source, (2) review and verify extractions, (3) compare verified knowledge to agent health and recalibrate.

The methodology treats operator judgment as a stabilizing mechanism in the current operating-state design. Across the three tasks, the operator connects sources, verifies candidate extractions,

compares review outcomes against agent-health signals, and recalibrates the system when those signals diverge. Human review functions as the boundary between candidate extraction and verified organizational memory. At the same time, the design process exposes the infrastructure needed for future versions of the system: richer observability, more precise telemetry, automated correction loops, and system architectures that make agent behavior inspectable and recalibratable at scale.

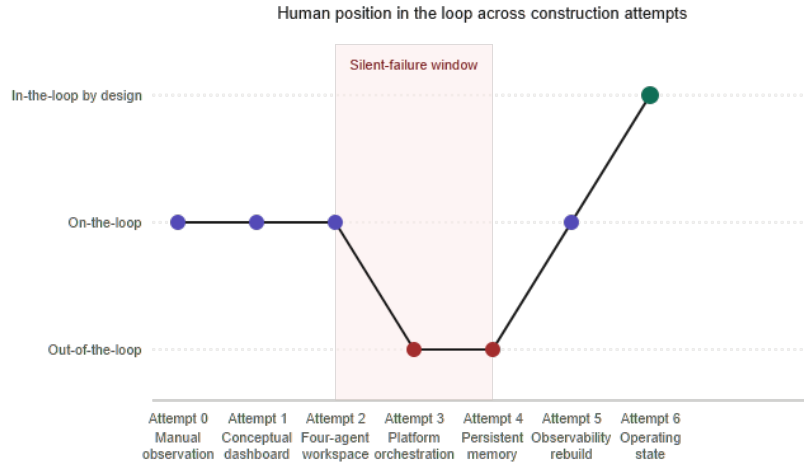


Figure 2: The operator’s position in the automation loop across the three tasks. Across the project’s construction history, the operator’s position in the loop drifted out, was recovered, and was finally placed structurally in the loop at Attempt 6, where verification is the action that pushes data into the verified library. The three tasks described in Sections 3.1.1 through 3.1.3 all operate inside this final structural position.

3.1.1 Task 1: Connect a Source

Purpose

The operator connects a source so the agents can extract candidate knowledge from it. A source is any digital record holding knowledge the operator wants captured: a code repository on GitHub, a folder on Google Drive, a database or workspace on Notion, or a channel on Discord.

Steps

1. Open Connected Sources.
2. Choose source type.
3. Provide source identifier.
4. Set refresh cadence.
5. Submit for verification.
6. Confirm source verified.
7. Submit source to the Ingestion Queue.
8. Confirm queue position.

Table VI: Task 1 (Connect a Source): information flow at the user-system interface, system feedback, and completion cue.

Field	Content
Information flow	The operator provides the source type, source identifier, and refresh cadence. The system verifies access, records the source, and places it in the Ingestion Queue.
System feedback	The system displays verification status (verified, verifying, or has issues) and queue position.
Completion cue	The task is complete when the source is verified and registered in the queue. Extraction begins when the queue reaches the source; the operator does not control the start time.

3.1.2 Task 2: Review and Verify Extractions

Purpose

The operator decides which of the candidate extractions produced by the agents are accurate enough to enter the verified Knowledge Library. The verified library contains only what the operator has reviewed and approved.

Steps

1. Open the Review Queue.
2. Select a candidate extraction.
3. Read the extracted evidence.
4. Compare the evidence to the source.
5. Decide disposition (Approve, Approve with corrections, Reject, or Mark as already in library). For Approve with corrections and Reject, choose a disposition reason from the structured set.
6. Submit decision.

Table VII: Task 2 (Review and Verify Extractions): information flow at the user-system interface, system feedback, and completion cue.

Field	Content
Information flow	The agents present a candidate extraction with its evidence and metadata; the operator reviews both, applies a disposition, and submits the decision. The system records the decision and updates the verified library.
System feedback	The candidate is removed from the Review Queue after the decision is submitted. Approved candidates and approved-with-corrections candidates enter the live knowledge graph; the graph processing runs and forms relationships among verified extractions.
Completion cue	The task is accomplished one candidate at a time. The verified Knowledge Library grows by the count of approved-and-approved-with-corrections candidates across the run.

Task 2 uses the candidate-level disposition gate. Each candidate passes through data verification and metadata verification before the operator selects a disposition. The disposition lever and options are summarized in Figure 3 and Table IX.

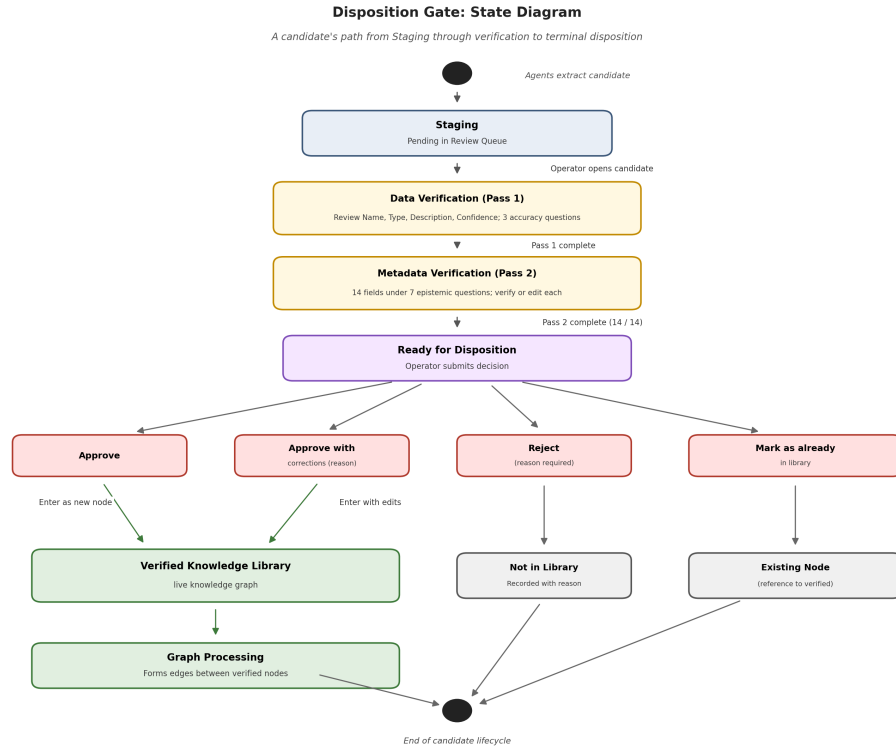


Figure 3: State diagram of the disposition gate. The candidate moves from Staging through two verification passes (Data Verification, then Metadata Verification), then through the operator’s disposition submission, to one of four terminal destinations. Approve and Approve with corrections enter the Verified Knowledge Library as new nodes, where graph processing then forms edges between verified nodes. Reject and Mark as already in library do not add to the library. Disposition reasons are required on the two transitions that involve operator judgment about why a candidate failed or needed correction.

3.1.3 Task 3: Compare Verified Knowledge to Agent Health and Recalibrate

Purpose

After candidate extractions have been reviewed, the operator compares the verified knowledge graph and review outcomes against the agent-health signals shown in the Agents area. The purpose is to determine whether the agentic system is calibrated or whether the operator needs to adjust tooling, ontology, prompting, or orchestration.

Steps

1. Review the verified knowledge graph after a review cycle.
2. Review disposition outcomes from the Review Queue, including approvals, approvals with corrections, rejections, duplicates, and disposition reasons.
3. Review agent-health signals, including confidence, trust, oversight, and run-status indicators.
4. Compare human judgment signals against agent signals. Human judgment is reflected in approval rates, correction patterns, rejection reasons, metadata-error patterns, and graph completion. Agent signals are reflected in confidence and health metrics.
5. Decide whether the system is calibrated.
6. If the signals align, continue using the current configuration.
7. If the signals diverge, recalibrate the relevant control lever: agentic tooling, ontology, prompting, or orchestration.
8. Run the next extraction cycle and compare again.

Table VIII: Task 3 (Compare Verified Knowledge to Agent Health and Recalibrate): information flow at the user-system interface, system feedback, and completion cue.

Field	Content
Information flow (system to operator)	The system shows the verified knowledge graph, review outcomes, disposition patterns, metadata-error patterns, and agent-health metrics.
Information flow (operator to system)	The operator interprets whether the human judgment signals and agent-health signals align. If they do not align, the operator changes tooling, ontology, prompting, or orchestration.
System feedback	The system shows whether recalibration improved the next extraction cycle. Improvement appears when agent confidence, operator approval patterns, correction rates, rejection reasons, and graph completion begin to converge.
Completion cue	The task is complete when the system reaches a calibrated steady state: agent confidence and health indicators match the operator’s review outcomes closely enough that the operator can continue the run without changing the system configuration.

Task 3 uses the system-level calibration gate. The calibration levers and options are summarized in Figure 4 and Table IX.

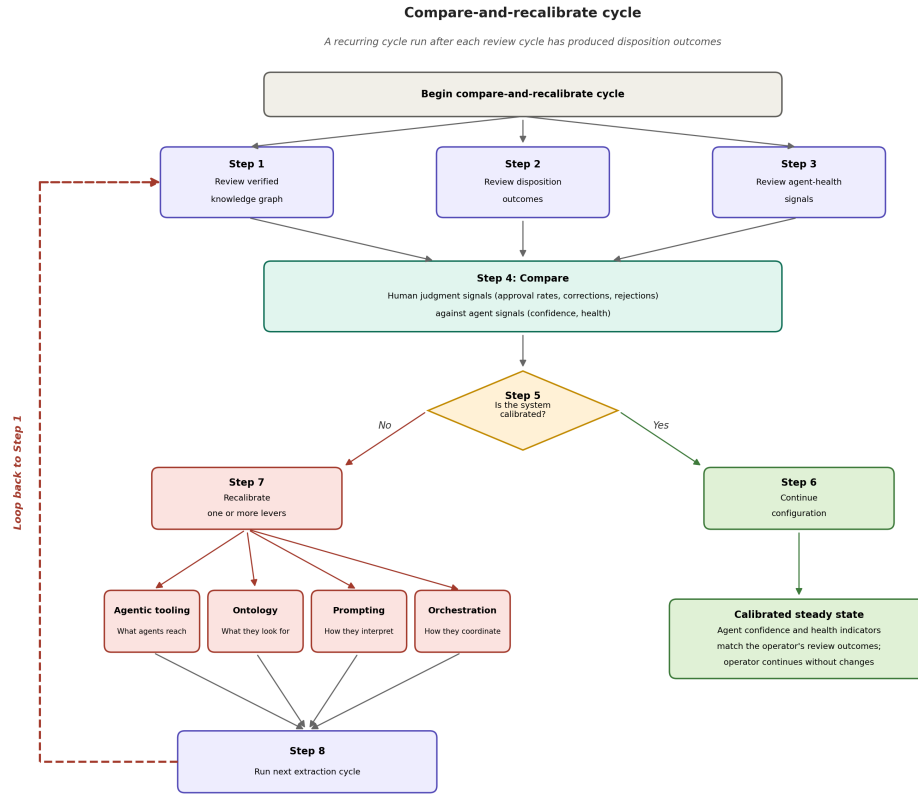


Figure 4: Task 3 compare-and-recalibrate cycle. The operator reviews the verified knowledge graph, disposition outcomes, and agent-health signals after each extraction cycle. If the human judgment signals (approval rates, correction patterns, rejection reasons) and the agent-health signals (confidence, trust, oversight) align, the operator continues the run with the current configuration. If they diverge, the operator recalibrates one or a combination of four control levers: agentic tooling, ontology, prompting, or orchestration. The next extraction cycle runs and the loop repeats until the system reaches a calibrated steady state where agent confidence and health indicators match the operator’s review outcomes.

Connect a source loads the queue. Review extractions empties the queue back into the verified

library at human pace. Compare verified knowledge to agent health and recalibrate when the two diverge.

3.2 Decision Gates Across Three Levels

Decision levers are found at Task 2 and Task 3 of the user journey. Task 2 represents a candidate-level decision lever where the operator decides what happens to one extraction. Task 3 represents a system-level decision lever where the operator decides whether the agentic system remains calibrated after review, while the system design has a project-level decision lever where infrastructure shortfalls force a decision about how the architecture should change.

Table IX: Decision gates across three levels: the evidence each gate uses, the options the operator chooses among, the lever the operator pulls, and the outcome the decision produces.

Level	Decision gate	Evidence used	Decision options	Lever used	Outcome
Candidate	Disposition gate	Candidate extraction, source evidence, data verification, metadata verification	Approve, Approve with corrections, Reject, Mark as already in library	Disposition	Verified node, corrected node, excluded candidate, or duplicate not added
System	Calibration gate	Verified graph, disposition outcomes, correction patterns, rejection patterns, metadata-error patterns, graph completion, agent-health signals	Tooling, Ontology, Prompting, Orchestration	Recalibration	Next extraction cycle runs under revised conditions
Project	Iteration gate	An infrastructure element no longer supports the users' needs; the gap is structural rather than situational	Reframe, Build, Replace	Design iteration	Next attempt revises the infrastructure to support the unmet need

The candidate-level gate is elaborated in Section 3.1.2 with Figure 3 (disposition state diagram). The system-level gate is elaborated in Section 3.1.3 with Figure 4 (compare-and-recalibrate cycle). The project-level gate is elaborated in Section 3.4.1 with Table X and Figure 7 (iteration gate timeline). The human-in-the-loop framing that holds all three gates together is visualized in Figure 2.

3.3 User-System Interaction Model

The operator and the application work together through a fixed set of screens and a recurring back-and-forth across them. Two figures describe this. Figure 5 lays out the screens of the application and how they connect. Figure 6 shows the conversation between the operator and the application across one full work session.

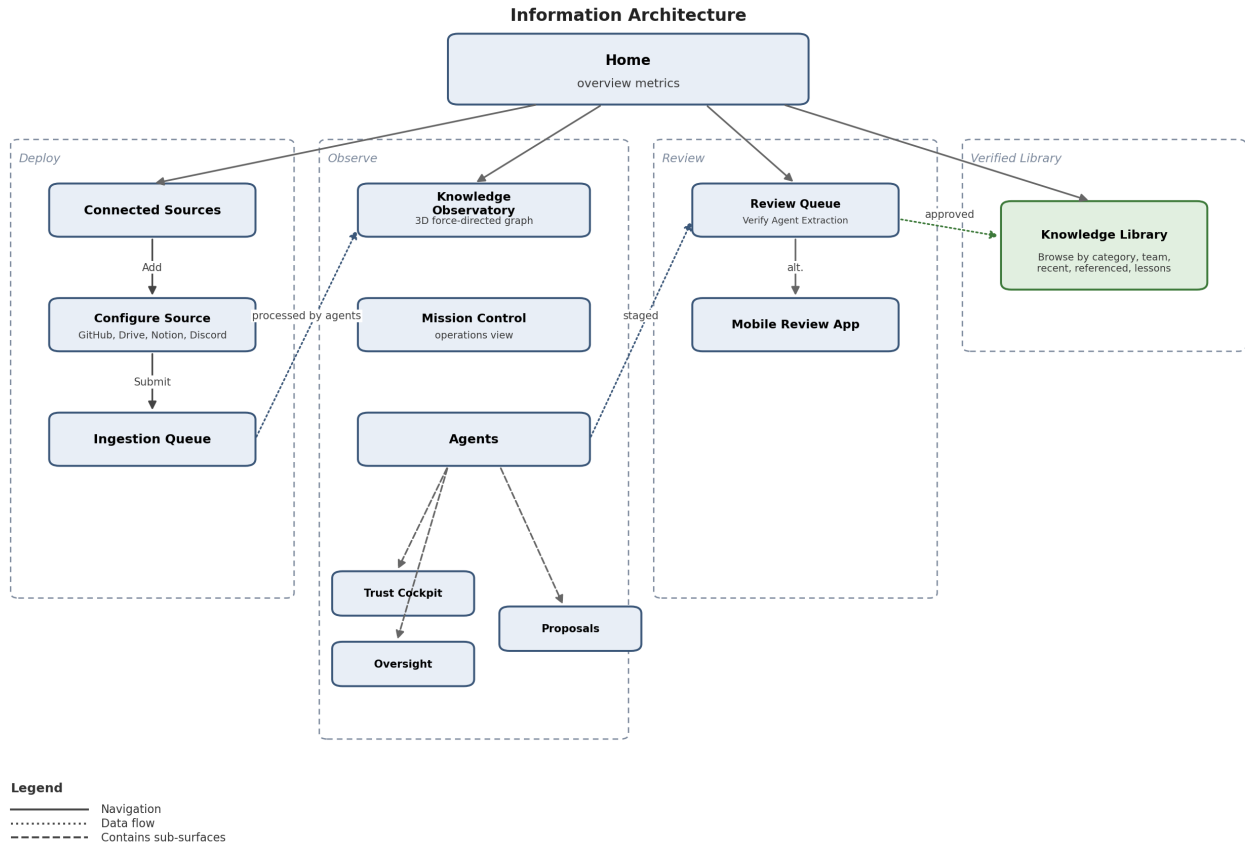


Figure 5: Information architecture of the application’s surfaces. Home is the entry point. Connected Sources, Configure Source, and Ingestion Queue serve the connect-a-source task. Review Queue and Mobile Review App serve the review-and-verify task. Knowledge Observatory, Mission Control, and the Agents area (with sub-surfaces Agent Trust Cockpit, Agent Oversight, and Agent Proposals) serve the compare-and-recalibrate task. Knowledge Library is the verified library. Solid lines show navigation; dotted lines show data flow between surfaces.

The screens are grouped by which of the three tasks they serve. Connect-a-source screens are used one after another in order. Review screens are used one candidate at a time. Compare-and-recalibrate screens are used freely, in any sequence, while the agents work and after review cycles. The dotted lines on the figure trace the data path through the system: a source submitted to the Ingestion Queue is picked up by the agents, written into the Knowledge Observatory, surfaced as a candidate in the Review Queue, and crossed into the Knowledge Library after the operator approves it.

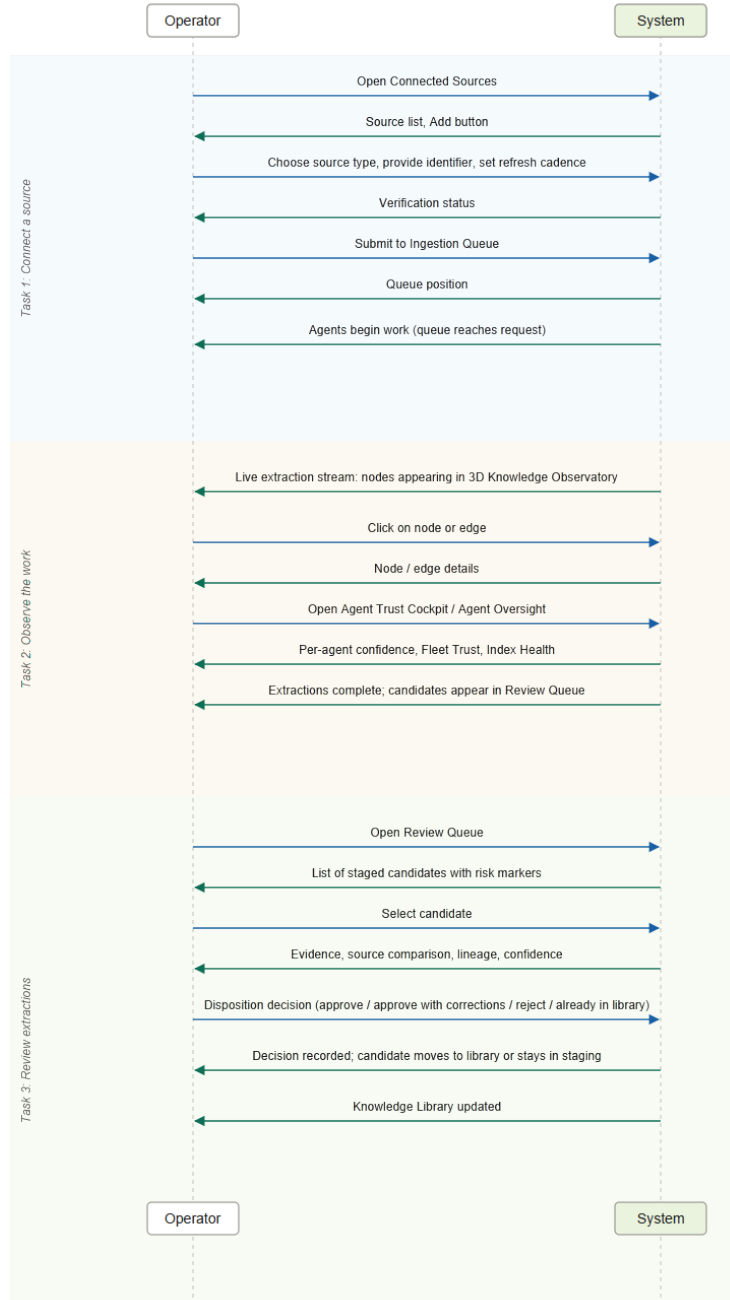


Figure 6: Sequence diagram of one full work session, with operator and system swim lanes. Operator messages travel left to right; system messages travel right to left. The three tasks are color-banded so the reader can follow the conversation through the session. In Task 1 (Connect a Source), the operator gives instructions and the system confirms them. In Task 2 (Review and Verify Extractions), the operator and the system work one candidate at a time, with the operator deciding the disposition and the system recording the decision and updating the Knowledge Library. In Task 3 (Compare Verified Knowledge to Agent Health and Recalibrate), the operator compares review outcomes against agent-health signals and adjusts the system configuration when the two diverge.

The shape of the conversation changes across the three tasks. In Task 1 the operator is giving instructions and the application is confirming them. In Task 2 the operator and the application work one candidate at a time: the operator decides, the application records and updates the Knowledge Library. In Task 3 the operator compares the review outcomes against agent-health signals and adjusts the system configuration when the two diverge.

3.4 Project Design

3.4.1 Iterative Design Process

The final design was developed through six iterative attempts. Each attempt tested a different arrangement between the human operator, the technical sources, and the agentic workflows intended to support knowledge capture. The design did not begin as a finished architecture. It emerged through repeated attempts to reduce the mismatch between rapid technical knowledge generation and the human ability to capture, verify, and reuse that knowledge.

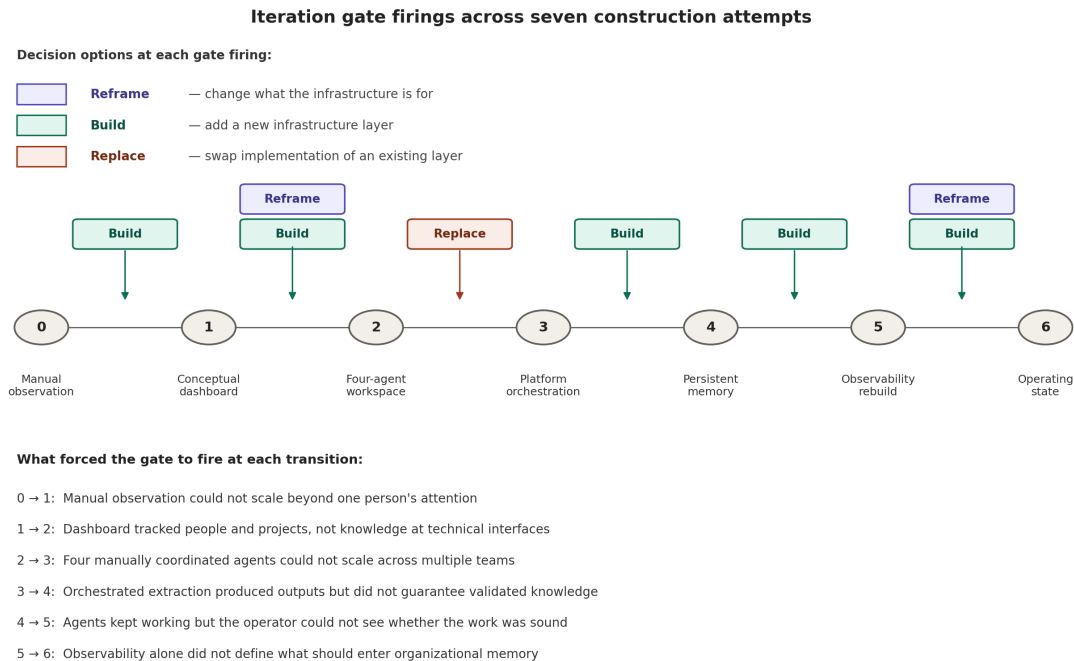


Figure 7: The iteration gate firing across seven construction attempts. At each transition between attempts, the operator chose one or a combination of three decision options described in Section 3.2 Table IX: Reframe (change what the infrastructure is for), Build (add a new infrastructure layer), or Replace (swap implementation of an existing layer). Most transitions fired a single option; the 1 to 2 transition fired Reframe and Build together when the system was retargeted from organizational visualization to knowledge-at-technical-interfaces and the agentic workspace was introduced; the 5 to 6 transition fired Reframe and Build together when the human's role was reframed from output verifier to structural gatekeeper and the verification gate, live graph, and bifurcated storage were built. The result of the 5 to 6 transition is the in-the-loop position visualized in Figure 2.

The design process is summarized in Table X. Each attempt exposed a different human-system problem. The following attempt changed the system architecture or interface to address that problem.

Table X: Iterative design process: each attempt’s arrangement, the issue it exposed, and the design change that produced the next attempt.

Attempt	Arrangement and HITL position	Issue exposed and design change produced
Attempt 0: Manual observation	Human-only observation of one space lab and its knowledge-transfer problems. The human is the observation system.	Knowledge retention depended on the continued presence of specific people; no instrumented way to capture what the lab was learning. Established knowledge retention as the central design problem.
Attempt 1: Conceptual dashboard	A Flask-based dashboard visualized universities, teams, projects, and collaborations. On-the-loop: the human did the conceptual work; the dashboard made the conceptual work visible.	The dashboard made the organization visible, but tracked people and projects rather than the knowledge held at technical interfaces. Shifted the design focus from participant tracking to structural knowledge mapping.
Attempt 2: Four-agent workspace	Four named agents worked with the human in a shared file workspace and Notion board. On-the-loop, direct in the repository: the human read what the agents wrote and corrected them by editing the shared file.	Direct human supervision worked at small scale, but four manually coordinated agents could not support expansion across multiple university teams. Introduced the need for stronger agent orchestration and scalable coordination.
Attempt 3: Platform orchestration	LangChain and LangGraph coordinated agent workflows across source documents. Out-of-the-loop on agent reasoning, on-the-loop for chart-production review.	Agents could produce structured outputs, but output production did not guarantee validated knowledge. Separated agent capability from knowledge validity and created the need for persistent memory and verification.
Attempt 4: Persistent memory and self-correction	A persistent-memory layer supported agent work across runs and allowed agents to produce analytical artifacts. Out-of-the-loop: the agents kept working and the human could not see whether the work was sound.	The system produced useful artifacts, but silent failures occurred because the human could not see enough of the agent workflow. Exposed observability as a design requirement, not an optional monitoring feature.
Attempt 5: Observability rebuild	LangSmith, OpenTelemetry, and Grafana were added to trace agent behavior and system activity. Returning on-the-loop: the human regained visibility into agent reasoning, agent boundaries, and aggregate behavior.	Visibility improved, but observability alone did not define what should enter the organizational memory. Led to the verification gate and operator-facing review structure.
Attempt 6: Operating-state prototype	The final prototype used source connectors, agentic extraction, bifurcated storage, a two-pass human verification gate, and a live knowledge graph. In-the-loop by design: the human cannot be bypassed; verification is the action that pushes data into the graph.	The pilot ended before systematic user testing could measure long-term effectiveness. Produced the final design for this project and the basis for future evaluation.

Across the six attempts, the design moved through three major shifts. First, the project moved from observing knowledge loss to visualizing the organization that produced it. Second, it moved from manually supervised agents to orchestrated agent workflows. Third, it moved from agent output production to human-verified knowledge mapping. The analytical artifacts the agents produced in Attempts 3 and 4 (team-boundaries reports, knowledge-gaps reports, transitive-chains traces) demonstrated that the system was technically feasible: the agents could work at scale across source documents, produce structured outputs, and self-correct. Feasibility cleared the way for the harder problem to surface: how to extract the types of knowledge most likely to be lost, one of the system’s binding limitations to overcome. The system had to capture embodied knowledge that cannot be written down in the form documentation assumes.

The iterative process showed that the main design problem was not how to automate extraction. The main design problem was how to make automated extraction accountable to human judgment without slowing the operator enough that the system would be abandoned. The final design addresses that problem by separating candidate extraction from verified knowledge and placing the operator at the disposition gate between them.

3.4.2 Final Operating-State Design

Operating-state architecture (Attempt 6): layered stack with observability spanning all layers

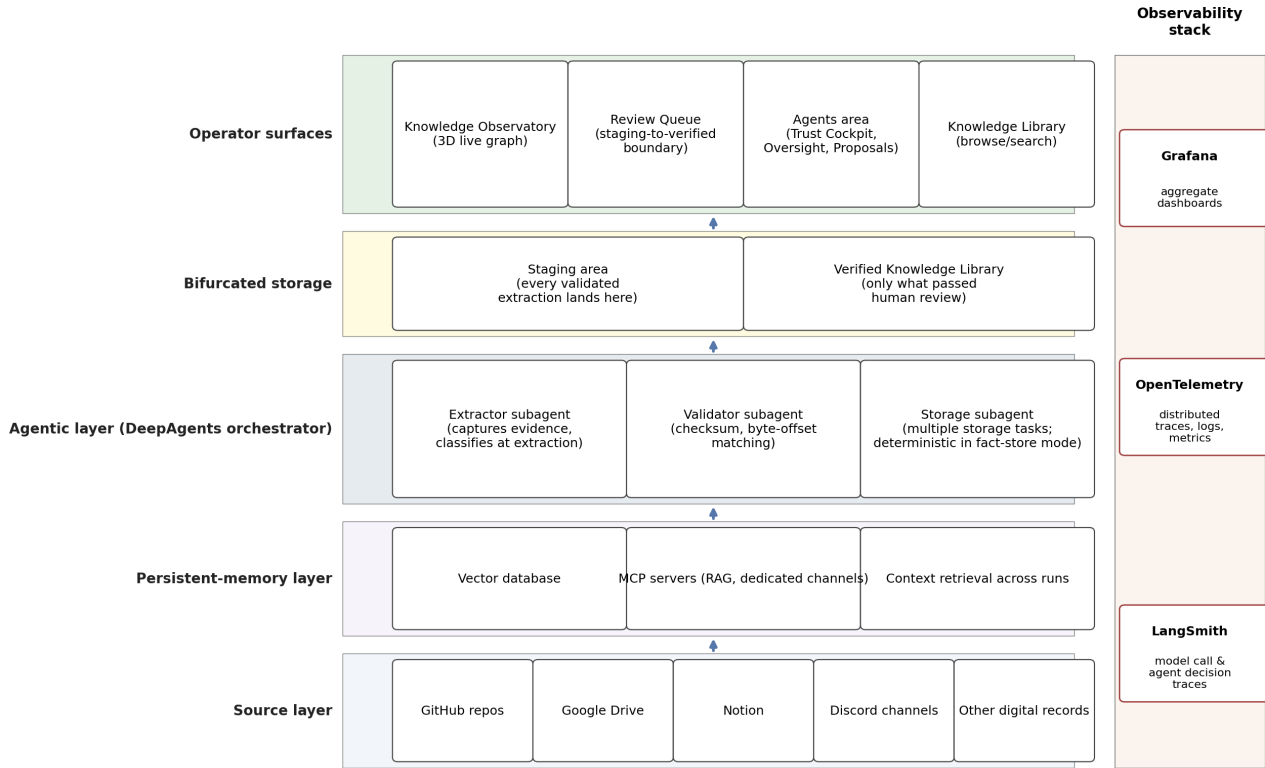


Figure 8: The operating-state architecture as a layered stack. From the bottom up: source layer (GitHub repos, Google Drive, Notion, Discord channels), persistent-memory layer (vector database, MCP servers for RAG), agentic layer (DeepAgents orchestrator coordinating Extractor, Validator, and Storage subagents), bifurcated storage (staging area and verified Knowledge Library), and operator surfaces (Knowledge Observatory, Review Queue, Agents area, Knowledge Library). The observability stack (LangSmith, OpenTelemetry, Grafana) spans all layers on the right.

The operating-state architecture has two parts doing different work. The agentic layer, the persistent-memory layer, the storage, the observability stack, and the operator surfaces are infrastructure: vendor-agnostic and replaceable. The mapping (where the system looks for knowledge) and the metadata analysis (how the system describes what it finds) are not vendor-agnostic. They are what makes the verified knowledge graph mean something. The infrastructure exists so the mapping and the metadata research can run.

Table XI: System architecture of the final operating-state design: each layer’s function and the implementation chosen for the pilot.

Component	What it does	Implementation
Source layer	The set of digital records from which the agents extract.	GitHub repositories (PROVES core, F Prime, Open Source Space Foundation); Google Drive, Docs, Sheets; Discord bot; Resend for email; Anthropic API; OpenAI optional.
Persistent-memory layer	A vector database accessed through dedicated MCP servers, holding the context the agents reach into across runs.	Docker-deployed FastMCP server (RAG) with a source registry; sentence-transformers for embeddings; pgvector for similarity search; tree-sitter for AST parsing in the risk scanner.

Continued on next page

Table XI continued from previous page

Component	What it does	Implementation
Agentic layer	An orchestrator coordinating three task-scoped subagents (Extractor, Validator, Storage) running extraction, validation, and storage in sequence.	DeepAgents middleware on LangGraph and LangChain; Claude Sonnet 4.5 as primary model; Claude Haiku for Validator and Storage subagents; FastAPI extraction API on Python 3.12.
Storage architecture	Storage is bifurcated. Every validated extraction enters a staging area; only extractions that pass human review enter the verified library [4]. The verified library is a live knowledge graph that receives each verified extraction as the operator submits a disposition; graph processing then runs to find relationships and form links. Nodes without yet-found relationships float in their own space until later verifications produce the connections.	Supabase Postgres with pgvector (42 migrations applied); bifurcated staging and verified library; LangGraph checkpoint store for agent state.
Classification policy	Classification happens at extraction, because once an entity is captured without its type, downstream agents cannot recover the correct one.	Enforced through the agent contracts, which bind each agent's behavior to the ontology at runtime.
Capture policy	Capture is exhaustive at the system level, because the agents do not assign criticality.	Enforced through the agent contracts, which bind each agent's behavior to the ontology at runtime.
Operator surfaces	The Knowledge Observatory is the dashboard the operator works against. The review surface is where the operator's judgment becomes the action that pushes verified data into the live graph.	React and TypeScript curation dashboard on Vercel; Tailwind CSS; Radix UI primitives; 3D force-graph and Three.js for the live knowledge graph; Mission Control, Library, and Admin surfaces; Notion webhook handler with auto-promote cron.
Observability stack	Spans all layers; carries traces of model calls, agent decisions, distributed traces across agent boundaries, and aggregate dashboards across the running system.	LangSmith in production for agent decision traces; OpenTelemetry and Grafana retained from Attempt 5, still feeding information to learn what production telemetry should be for this kind of work.
Hosting / deployment	Where each layer runs.	Vercel (curation dashboard and Notion cron); Netlify (Notion webhook function); Docker self-hosted (Extraction API, MCP server); Supabase managed (Postgres with pgvector).

The six screenshots that follow show the operator surfaces in the deployed pilot. They are the operating-state design rendered at the surface the operator's hand actually meets.

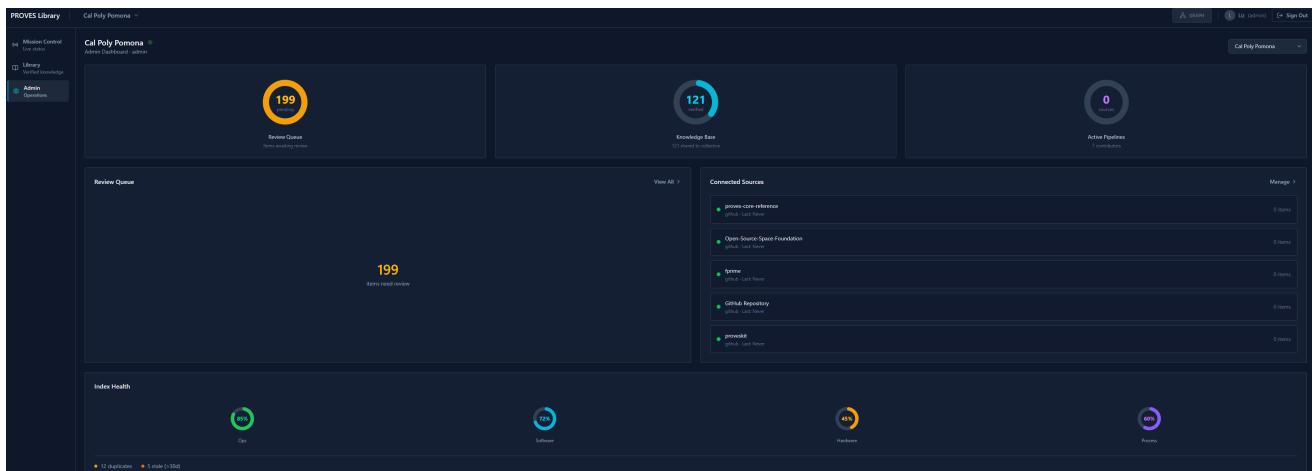


Figure 9: The admin operations dashboard for the Cal Poly Pomona workspace. The operator sees the Review Queue with 199 items pending, the Knowledge Base with 121 verified items, and Connected Sources listed by name. The Index Health gauges at the bottom (Ops 85%, Software 72%, Hardware 45%, Process 60%) show coverage by domain across the verified library. This is where the operator opens a session and decides where to spend attention.

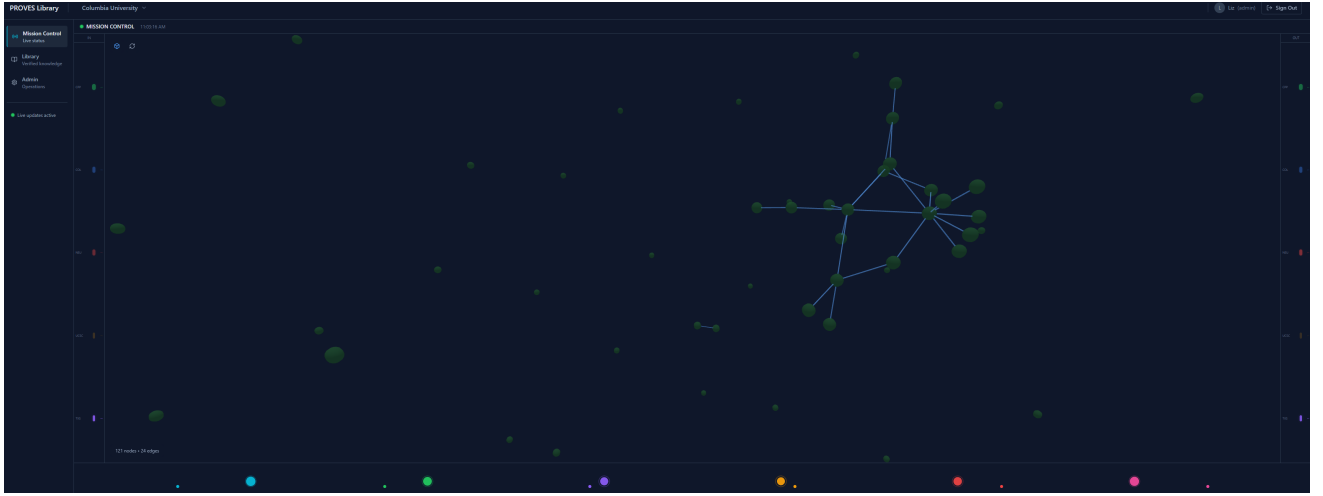


Figure 10: The Mission Control view at 11:03 AM showing the live three-dimensional knowledge graph. The y-axis labels mark the contributing universities (CPP, COL, NEU, UCSC, TXS). 121 nodes and 24 edges. One connected cluster sits on the right where graph processing has found relationships among verified extractions. Many nodes float without connections, waiting for later verifications to produce overlapping coordinates. The graph is live; nodes enter and connections form as the operator submits dispositions.

Item Name	Type	Date	Confidence
MakeCommand to BuildSystem	dependency	1/10/2023	91%
ScreenCommand to SerialConsoleAccess	dependency	1/10/2023	91%
SerialConsoleInterface	component	1/10/2023	91%
RoundRobinPort to CodeValidation	component	1/10/2023	91%
RoundRobinSystem	component	1/10/2023	91%
LinuxOS to DevEnvironment	dependency	1/10/2023	91%
PythonC2Client to DevWorkflow	dependency	1/10/2023	91%
LinuxDevEnvironment	component	1/10/2023	91%
dependenciesystem hardware to base_stack	dependency	1/10/2023	80%
componentPlatform_Squared_Ratio	component	1/10/2023	91%
componentPlatform_Squared_2_Ratio	component	1/10/2023	91%
constraintUser_abstraction_boundary	dependency	1/10/2023	91%
dependenciespy to function.py	dependency	1/10/2023	70%
dependenciesfunction.py to baseModule_Semantic	dependency	1/10/2023	71%
componentbaseModule_Semantic	component	1/10/2023	91%
componentfunction.py	component	1/10/2023	91%

Figure 11: The Review Queue showing pending items for the Cal Poly Pomona workspace. Each row is a candidate extraction (dependencies, components, constraints) from the *proveskit* repository, dated, typed, and shown with agent confidence (70% to 95%). The operator works through these one at a time.

Extractor vs Human Training Data

Left: What the AI extracted. Right: Your corrections (or confirm if correct).

AI Extractor Found
Name: `componentFpConfig.h`
Type: `component`
Description: `Project-specific configuration for framework and GDS types`
Confidence: NaN%

Your Corrections
Name: `componentFpConfig.h`
Description: `Project-specific configuration for framework and GDS types`
Leave blank if extractor was correct. Only fill in corrections.

Evidence from Source View Original

"Project may configure the framework types that the framework and components use for implementation through FpConfig.h. The default configuration as supplied with F uses the above platform types where applicable."

Is this accurate?

- ☐ **Does this actually exist?**
Is this real, not hallucinated?
- ☐ **Does the evidence support this?**
Does the quote say what the AI claims?
- ☐ **Are the details correct?**
Names, interfaces, behavior?

(a) Data verification pass.

Knowledge Metadata Verify Agent Extraction 0/14 Verified

Review what the AI extracted. Click ✓ to verify or ✗ to correct.

Q1: Who knew this?
Source Type: `human`
Contact Mode: `derived`

Q2: Where does this live?
Storage: `externalized`
Signal Type: `spec`

Q3: What stays connected?
Dependencies: `componentPlatformTypes.h, componentBasicTypes.hpp`
Role in Sequence: `none`

Q4: When is this true?
Assumptions: `Not detected`
Scope: `system`

Q5: How current is this?
Staleness Risk: `30%`
Refresh When: `Not detected`

Q6: Who wrote this?
Intent: `instruct`
Uncertainties: `Interaction between FpConfig.h and PlatformTypes.h override precedence not fully specified`

Q7: Need to DO it?
Needs Practice: `Not detected`
Transferability: `portable`

⚠ Answer the accuracy questions above

⊗ Reject

(b) Metadata verification pass.

Figure 12: The two-pass verification gate. (a) The data verification surface: AI Extractor Found on the left, Operator corrections on the right, the source quotation between them, and three accuracy questions at the bottom. (b) The metadata verification surface: seven epistemic questions, each carrying two fields, with the 0/14 Verified counter at the top.

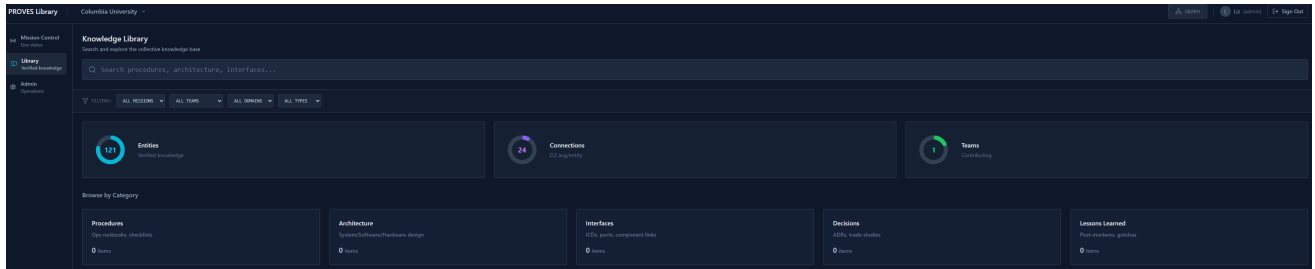


Figure 13: The Knowledge Library viewed from the Columbia University workspace. 121 entities, 24 connections, 1 contributing team visible from this workspace. Browse-by-Category navigation across Procedures, Architecture, Interfaces, Decisions, and Lessons Learned. The library is the same verified knowledge graph the Cal Poly Pomona operator builds, viewed from another organization's seat.

The final design treats the operator as part of the system architecture. The agents extract candidate knowledge, but the operator determines whether the candidate becomes verified knowledge. The system does not depend on the operator watching every agent action in real time. Instead, it gives the operator structured review surfaces, source evidence, disposition options, and observability tools so that human judgment enters at the point where it has the most effect.

3.4.3 Tools Used to Develop the Design

The tools used to develop the design are organized below by the design need each tool served. The implementations listed are the specific software and platforms chosen for the operating-state prototype. The design need each addresses is the load-bearing reference; the specific tool is a replaceable choice.

Table XII: Tools used to develop the operating-state design: the design need each one served and the specific implementation chosen for the pilot.

Design need	Tools used	Purpose
Interface design and screen progression	Figma, Vite, React, TypeScript	Built and refined dashboard surfaces
Source extraction and agent workflows	LangGraph, DeepAgents, Anthropic API, FastAPI	Coordinated extraction, validation, and storage agents
Persistent memory and retrieval	Supabase Postgres, pgvector, FastMCP server	Stored source context and supported retrieval across runs
Knowledge graph display	Three.js, 3D force-graph	Displayed verified entities and relationships
Observability	LangSmith, OpenTelemetry, Grafana	Traced model calls, agent decisions, and system activity
Deployment	Vercel, Docker, Supabase	Hosted dashboard, agent services, and database

These tools were not the research contribution by themselves. They were the implementation choices used to build the human-system arrangement tested in the project.

3.5 Design walkthrough and evaluation

The design walkthrough has four parts. The first compares each constraint identified in Section 2 against the design feature built to address it. The second names what the pilot deployment did and did not measure. The third names the real-world performance risks that remain after the design is in place. The fourth names the empirical evaluation methods that would measure how well the design supports users in sustained operation, once users can test the system over time.

3.5.1 Constraint-Based Walkthrough

Each design factor identified in Section 2 is paired below with the design feature that addresses it and the part of the system where the feature lives. The pilot status column names what the deployment did and did not yet exercise.

Table XIII: Design factors, the design feature that addresses each one, where in the system the feature lives, and the pilot status of each.

Design factor	Design feature that addresses it	Where in the system	Pilot status
Rapid knowledge generation	Extraction reads source artifacts (repositories, design documents, channels) at the moment work is happening; the verified library grows alongside the work rather than after the fact	Source connectors + Extractor subagent	Pilot-deployed, not yet user-tested
Distributed teams	Shared Ingestion Queue and shared Knowledge Library accessible across institutions; same library viewable from any participating workspace	Persistent-memory layer + bifurcated storage	Pilot-deployed, not yet user-tested
Highly technical content	Classification at the moment of extraction; the seven-question metadata framework; structured disposition reasons at the Review Queue	Extractor subagent + Review Queue	Pilot-deployed; metadata framework research in progress
Rapid development cycles	Pace mismatch absorbed by the staging-to-verified boundary; Mobile Review App designed for review in spare moments but not completed due to display parity issues with the desktop	Bifurcated storage + Mobile Review App	Staging boundary pilot-deployed; Mobile Review App designed, not completed
Role-differentiated access	Workspace-level segregation by institution and role; the same Knowledge Library presents different views to different audiences	Operator surfaces	Pilot-deployed, not yet user-tested
Cross-phase knowledge gaps	Verified library is relationally complete rather than purpose-built; relationships in the knowledge graph survive across project phases	Knowledge Library + live knowledge graph	Built; not yet observable
Onboarding and offboarding gaps	Continuous extraction captures work as it happens, not at the end; the verified library does not depend on a clean handoff window	Agentic layer + Ingestion Queue	Built; not yet observable
Lost technical context	Lineage and source evidence preserved on every entry; the seven metadata fields surface why decisions were made	Knowledge Library entries + metadata verification surface	Built; not yet observable
Organizational support across transitions	Six browse views (Category, By Team, Most Recent, Most Referenced, Coverage by Domain, Lessons Learned) make the library legible to non-operator audiences	Knowledge Library navigation	Built; not yet observable

3.5.2 Pilot Status

The operating-state prototype is deployed across the participating university workspaces with verified knowledge graphs visible to the operator, the next cohort, and the supporting organizations through the dashboard. The Cal Poly Pomona workspace at the time of writing holds 121 verified entities and 24 connections, with the Review Queue holding 199 pending candidates. The pilot demonstrated that the architecture runs end-to-end and that each design factor in Table XIII has a corresponding feature built into the system. The pilot did not measure operator workload over sustained sessions, situation awareness during agent observation, trust calibration between operator and agents, inter-rater agreement on disposition decisions, or the rate at which verified knowledge actually transfers across cohorts. Those measurements require users running the system at scale, over time, with the systematic data-gathering the pilot did not have budget or duration to support.

3.5.3 Real-World Performance Risks

The design addresses the constraints identified in Section 2, but its effectiveness in sustained operation depends on conditions outside the design itself. The risks below are the factors most likely to reduce the system’s effectiveness even when the design works as built.

Table XIV: Real-world performance risks: factors outside the design that could reduce its effectiveness, and the design responses already in place to mitigate each one.

Risk	Effect on the system	Design response
Operator does not review the queue consistently	Candidate knowledge accumulates in staging and does not enter the verified library; the library stops growing	Queue indicators showing pending count; consistent navigation across the three operator surfaces; Mobile Review App designed for review in spare moments
Agent extraction quality drifts	Review burden increases as the operator corrects more candidates; operator trust in the agents decreases	Confidence scores on each extraction; source quotation visible alongside; structured disposition reasons; LangSmith traces surface when an agent is drifting
Source access breaks	Agents stop extracting from the affected source; the library stops receiving current knowledge from it	Connected Sources surface shows verification status (verified, verifying, or has issues) so the operator can see when a source has broken
Supporting organization stops maintaining the library	New sources stop being added; existing entries become outdated; the library becomes stale and stops being useful to incoming cohorts	Refresh-cadence metadata on each source; staleness-risk metadata on each entry; the supporting-organization role explicitly named so the work has an institutional owner

3.5.4 Future Empirical Evaluation Methods

The constraint-based walkthrough shows that the design includes features addressing the identified constraints. It does not measure how well users perform with the system over time. The next evaluation phase would require empirical testing with operators and beneficiaries, framed by the human-centered design and usability standards in widespread use across software products that people work against: ISO 9241-11:2018 defines usability as effectiveness, efficiency, and satisfaction in a specified context of use [6]; ISO 9241-210:2019 defines the human-centered design process for interactive systems [7]; and ISO/IEC 25010:2023 provides a broader software quality model covering reliability, maintainability, security, portability, and interaction capability [8].

The methods named below come from the human factors research literature. Each is widely used in the field and has primary sources that define its use.

Table XV: Research methods that would provide measurement instruments for evaluating how well the design supports users in sustained operation, and the primary citation for each method.

Method	What it would measure for this system	Primary citation
NASA-TLX (Task Load Index)	Operator cognitive workload across six dimensions (mental, physical, temporal demand, performance, effort, frustration) during each of the three tasks. Measured after a representative work session to show whether the design keeps the operator’s workload within sustainable bounds [9], [10].	Hart and Staveland (1988); Hart (2006)
SAGAT (Situation Awareness Global Assessment Technique)	Operator situation awareness during Task 3 (compare verified knowledge to agent health and recalibrate). Measures whether the operator accurately perceives what the agents are doing, understands what it means, and can project what will happen next. Run by freezing a simulated session and querying the operator [11], [12].	Endsley (1988); Endsley (1995)
System Usability Scale (SUS)	Global subjective usability of the operator surfaces and the Knowledge Library, administered after sustained use. Produces a 0-to-100 score that is comparable against published benchmarks and across system versions [13].	Brooke (1996)
Lee and See trust framework	Trust calibration between the operator and the agents: whether the operator’s expressed trust matches the agents’ actual reliability, and whether reliance is appropriate (neither over-trust nor under-trust). Measured against the Agent Trust Cockpit displays and the Review Queue disposition decisions [14].	Lee and See (2004)
Cohen’s kappa	Inter-rater agreement on disposition decisions across multiple operators reviewing the same candidate set. Tests whether different operators reach consistent dispositions, which is the precondition for the Knowledge Library being relationally complete across institutions [15].	Cohen (1960)

The construction story is the strongest evidence available within the project’s scope that the design factors were addressed. Measurement of how well the design supports users in sustained operation is what the methods above would produce, in the next phase of the work.

4 Conclusion

4.1 What the project process showed

The project process showed that the architecture and the research inside it are doing different work. Most of the architecture is vendor-agnostic. The orchestration, persistent memory, storage, observability stack, and dashboard surfaces could each be implemented with different technology choices, and the system would still be the system. The mapping and metadata analysis are different. They are the research-bearing parts of the work. The mapping decides where in the system to look for knowledge. The metadata analysis decides how to describe each piece of knowledge so relationships between pieces can be found. Together, they are what make the verified knowledge graph mean something. Everything else in the architecture exists to support that work.

The dashboard is the surface that makes the work legible to the people who depend on it. The operator sees the live knowledge graph forming as verified data enters it. The next cohort sees what was collected and how it connects. The supporting organization sees what it is inheriting across cohort transitions. The dashboard is therefore not an operator-only surface. It is shared visibility across the people who create, inherit, and maintain the knowledge base.

4.2 What more time, money, and resources would have allowed

The mapping and metadata analysis are the parts of the work still in progress. The metadata analysis solidified into a working framework of seven epistemic questions at the end of the construction period, but the research needed to determine which questions are necessary, which are sufficient, and which need to be reformulated is ongoing. With more time, the work would continue less on the architecture itself, which is mature enough to support the next phase of research, and more on the mapping and metadata framework: testing the seven questions across more domains, identifying where they break, and refining them until they describe what knowledge does in a sociotechnical system without forcing a shape that does not fit.

With more money, the project would have supported production-scale observability and systematic evaluation across multiple reviewers. The evaluation would measure operator workload, review latency, disposition reason distribution, edit rate, queue depth over time, and agreement between reviewers. With more information, especially deeper access to organizational records from the participating university teams, the mapping research could compare what the system captured against what was actually lost or became difficult to recover during the same period. That comparison would give the framework the empirical anchor it still needs.

4.3 How the project design process would be improved

The design process would be improved by rebuilding the relationship between the agents and the system from the inside out. The current operating-state design works at the scale of one program because observability bridges the gap between what the agents are doing and what the

human can verify. At larger scale, that bridge would not be enough. Different parts of the system would need different forms of observability, different telemetry for different signals, and different agentic supports for different types of human work.

The next design process would therefore begin with a more deeply embedded architecture. The repository, hardware, software, data, and internal tools would be designed to describe themselves to the agents rather than requiring the agents to infer structure after the fact. In that version, the system would be self-describing in the ways the agents need it to be self-describing. Agents would encounter tagged structures, known interfaces, defined dependencies, and traceable sources rather than loose artifacts that must be interpreted from the outside.

The second improvement would be a distributed connection layer between teams. Each team or organization would maintain its own internal architecture, but the system would provide connective pathways between distributed-but-collaborating groups. This would allow teams to remain autonomous while still preserving the knowledge that crosses organizational boundaries. Each agent in that architecture would also need a different shape depending on the type of human work it supports, rather than using one general agent pattern with different prompts. Figure 14 shows the resulting structure: three teams each carrying their own deeply embedded internal architecture, connected by dispersed highways, with agents in different modes for different shapes of human work.

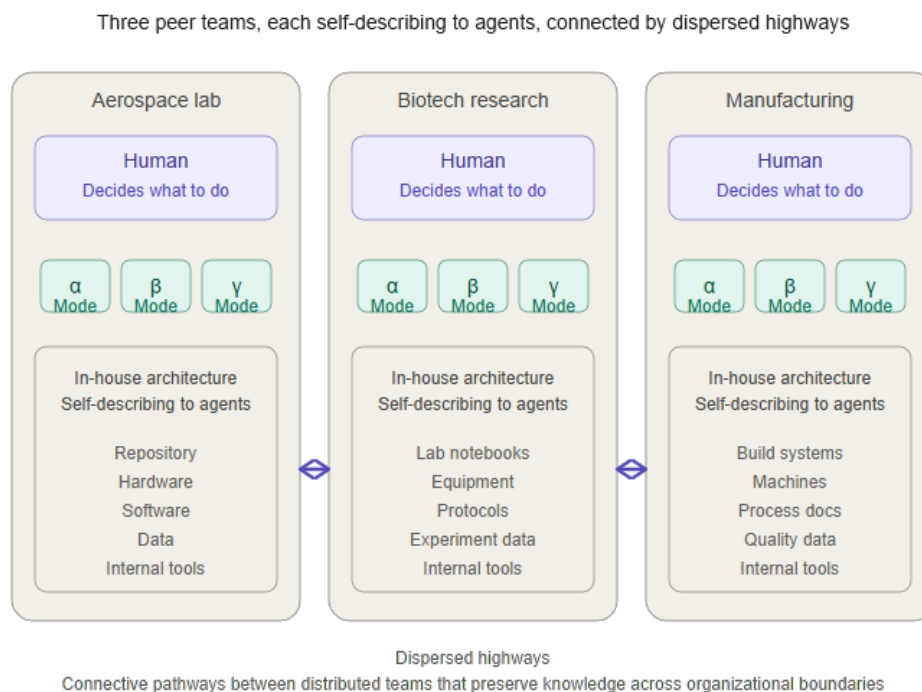


Figure 14: Architectural improvement vision. Three peer teams in different domains (aerospace, biotech, manufacturing) each carry the same three-layer internal architecture: a human at the top deciding what to do with what was found, agents in different modes (alpha, beta, gamma) shaped to different kinds of human work, and a deeply embedded in-house architecture that describes itself to the agents rather than requiring them to infer structure. Dispersed highways connect the three teams at the architecture layer, preserving knowledge across organizational boundaries while leaving each team’s local architecture autonomous.

The broader mapping system could then extend across domains where the same operating conditions apply: biotechnology research labs [5], manufacturing engineering teams, software

architecture groups, and other technical organizations producing knowledge faster than the people producing it can document. The sources would also extend beyond digital repositories to physical experiment data, scanned lab notebooks, design reviews, audio recordings, and video of work being done. Each new domain would test whether the metadata framework captures the right things. The framework would refine as the domains test it.

The shift in the human's work is the change the construction story has been pointing toward. In the current design, the human verifies that the agent understood. In the next design, the human would decide what to do with what was found. That is a different human-machine relationship from the one observability supports today.

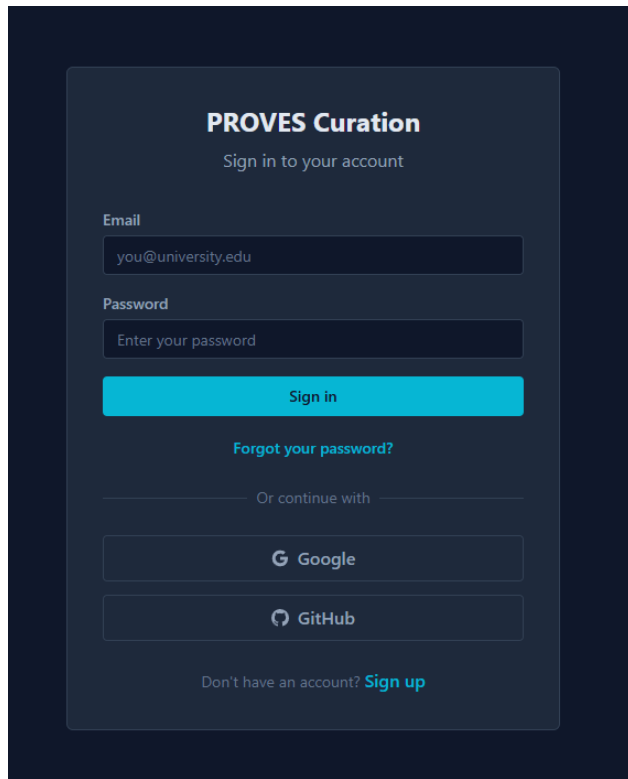
4.4 Closing

The mapping system was built to prevent catastrophic knowledge loss by locating knowledge at the structural interfaces where it lives, before the people who hold it move on. The pilot in PROVES preserved a structured record of knowledge that would otherwise have remained vulnerable to loss. The continuation, across different domains and different surfaces, would extend that preservation to places where knowledge is generated faster than the people producing it can document, and where the loss of that knowledge is difficult to measure because few systems are designed to observe it.

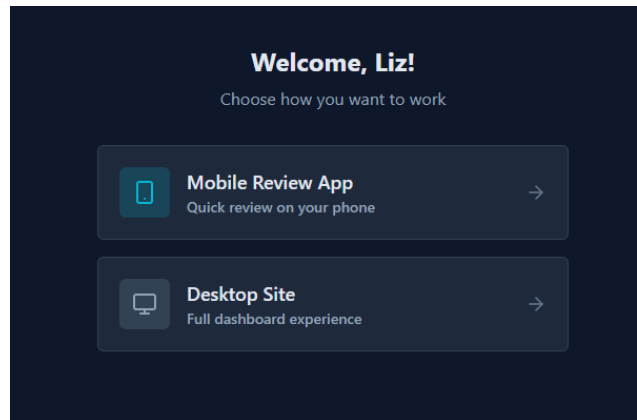
The architecture is the support structure that makes the work possible. The research is what will refine what the work captures. The dashboard is what makes that work legible to the people who depend on it. The work that continues is the research, because that is the part that will determine whether the next cohort, and the organizations that hold them stable, can read the system the previous cohort built.

Appendix A: FRAMES Prototype Screenshots

The following figures show the dashboard implementation built during the FRAMES (Framework for Resilience Assessment in Modular Engineering Systems) framing of the work, an early structural diagnostic for university CubeSat program resilience using Nearly Decomposable Architecture and Hybrid Autonomous Organization theory. The framing at this stage was organized around classifying modules and interfaces in the university-government-industry boundary cluster rather than locating knowledge at structural interfaces. The dashboard combined Flask and Three.js to make the organizational structure visible, but the analytical depth remained empty because the back-end could not yet support it, and the framing had not yet shifted toward tracking knowledge as a sociotechnical interface phenomenon. The three-dimensional collaboration network view was structurally prescient of what the operating-state Knowledge Observatory would later become.

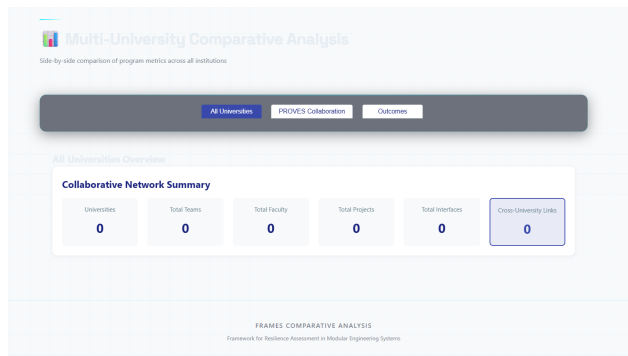


(a) Landing page (sign in).

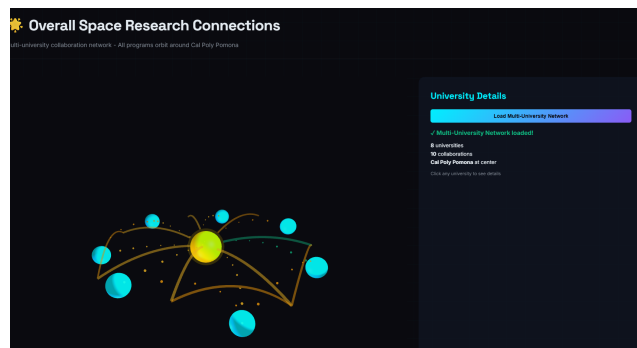


(b) University and role selection.

Figure 15: FRAMES dashboard, entry surfaces. (a) Landing page: “Space Program Knowledge Network,” framed around tracking teams, mentors, and projects. (b) Eight-university role selection portal with Cal Poly Pomona as lead and Faculty/Staff vs. Team Lead roles.



(a) Comparative analysis.



(b) 3D collaboration network.

Figure 16: FRAMES dashboard, analysis surfaces. (a) Multi-university comparative analysis with six count cards, all zero because no data had been entered. (b) Three-dimensional Overall Space Research Connections view, with Cal Poly Pomona at the center of an orbital network of eight universities. The 3D model was structurally prescient of the operating-state Knowledge Observatory.

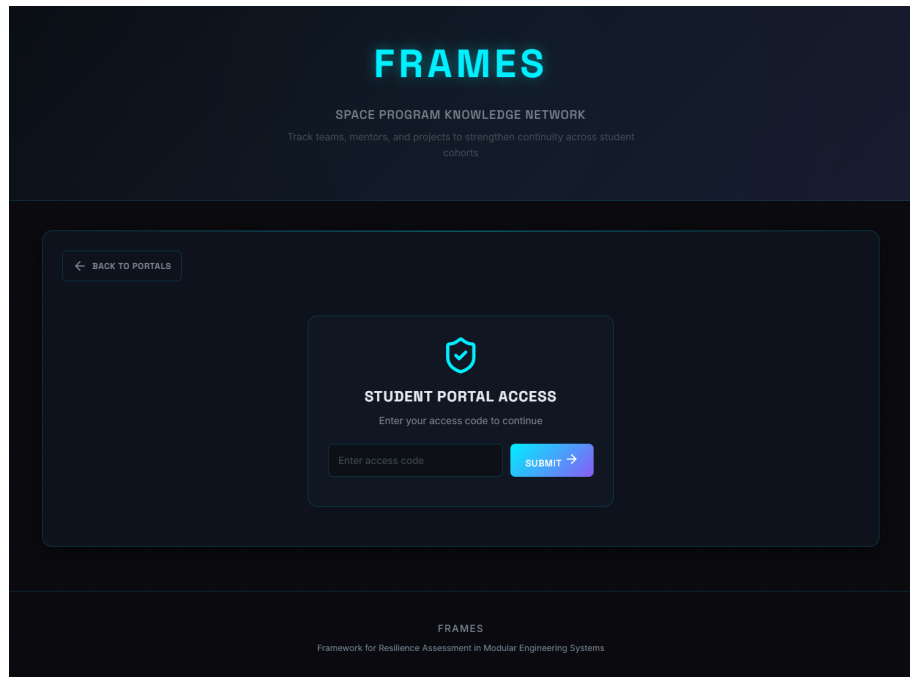


Figure 17: FRAMES Research Tools and Analysis surface. Six tool cards (Multi-University Network, Comparative Analysis, Energy Loss Analysis, Equilibrium Testing, Time-Series Analysis, Export and Reports); four were marked Coming Soon. Below, individual university networks for the eight participating institutions.

References

- [1] S. M. Noble, M. Mende, D. Grewal, and A. Parasuraman, “The Fifth Industrial Revolution: How harmonious human-machine collaboration is triggering a retail and service (r)evolution,” *Journal of Retailing*, vol. 98, no. 2, pp. 199-208, 2022, doi: 10.1016/j.jretai.2022.04.003.
- [2] A. Daghfous, N. T. Amer, O. Belkhodja, L. C. Angell, and T. Zoubi, “Managing knowledge loss: a systematic literature review and future research directions,” *Journal of Enterprise Information Management*, vol. 36, no. 4, pp. 1008-1031, 2023, doi: 10.1108/JEIM-05-2022-0171.
- [3] N. Levallet and Y. E. Chan, “Organizational knowledge retention and knowledge loss,” *Journal of Knowledge Management*, vol. 23, no. 1, pp. 176-199, 2019, doi: 10.1108/JKM-08-2017-0358.
- [4] E. Osborn, “Open Source CubeSat Library Database Schema for the Interface Anchored Knowledge Mapping System,” Zenodo, Apr. 26, 2026, doi: 10.5281/zenodo.19780470.
- [5] A. Jain, “How knowledge loss and network-structure jointly determine R&D productivity in the biotechnology industry,” *Technovation*, vol. 119, p. 102607, 2023, doi: 10.1016/j.technovation.2022.102607.
- [6] International Organization for Standardization, *ISO 9241-11:2018, Ergonomics of human-system interaction: Part 11: Usability: Definitions and concepts*. Geneva: ISO, 2018. [Online]. Available: <https://www.iso.org/standard/63500.html>

- [7] International Organization for Standardization, *ISO 9241-210:2019, Ergonomics of human-system interaction: Part 210: Human-centred design for interactive systems*. Geneva: ISO, 2019. [Online]. Available: <https://www.iso.org/standard/77520.html>
- [8] International Organization for Standardization and International Electrotechnical Commission, *ISO/IEC 25010:2023, Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): Product quality model*. Geneva: ISO, 2023. [Online]. Available: <https://www.iso.org/standard/78176.html>
- [9] S. G. Hart and L. E. Staveland, “Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research,” in *Human Mental Workload*, P. A. Hancock and N. Meshkati, Eds. Amsterdam: North-Holland, 1988, vol. 52, pp. 139-183, doi: 10.1016/S0166-4115(08)62386-9.
- [10] S. G. Hart, “NASA-Task Load Index (NASA-TLX); 20 years later,” *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 50, no. 9, pp. 904-908, 2006, doi: 10.1177/154193120605000909.
- [11] M. R. Endsley, “Situation awareness global assessment technique (SAGAT),” in *Proceedings of the IEEE 1988 National Aerospace and Electronics Conference*, Dayton, OH, 1988, pp. 789-795, doi: 10.1109/NAECON.1988.195097.
- [12] M. R. Endsley, “Measurement of situation awareness in dynamic systems,” *Human Factors*, vol. 37, no. 1, pp. 65-84, 1995, doi: 10.1518/001872095779049499.
- [13] J. Brooke, “SUS: A quick and dirty usability scale,” in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London: Taylor and Francis, 1996, pp. 189-194.
- [14] J. D. Lee and K. A. See, “Trust in automation: Designing for appropriate reliance,” *Human Factors*, vol. 46, no. 1, pp. 50-80, 2004, doi: 10.1518/hfes.46.1.50_30392.
- [15] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37-46, 1960, doi: 10.1177/001316446002000104.